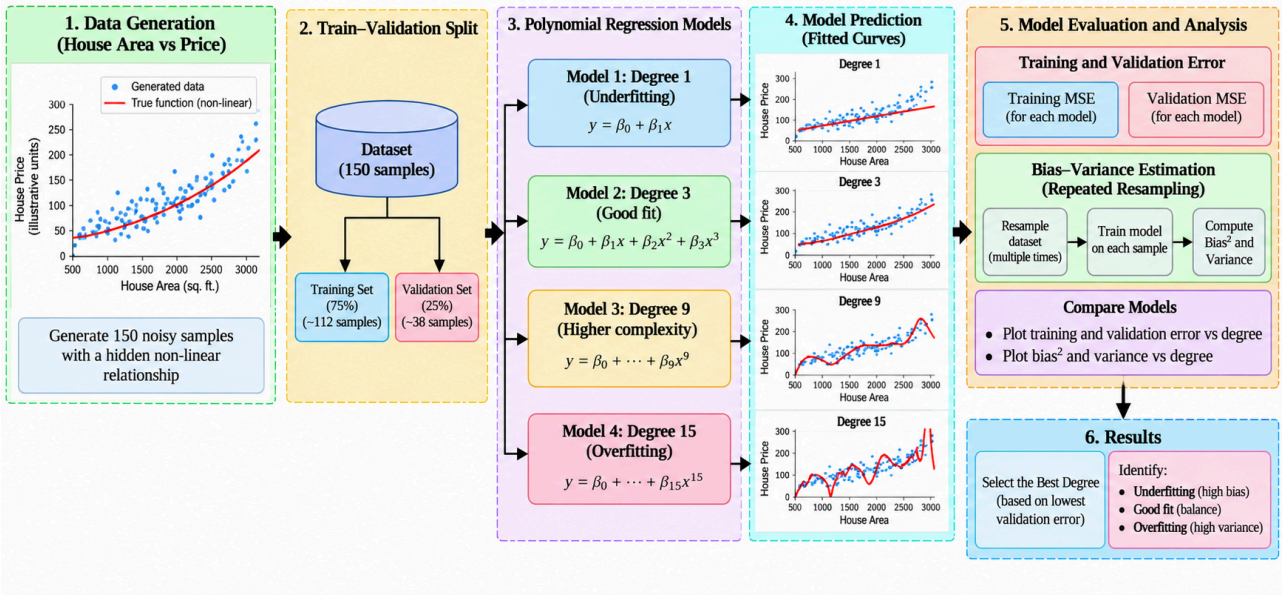
Experiment 5: Bias–Variance, Overfitting and Underfitting Analysis

1. AIM

To demonstrate underfitting, good fit, and overfitting using polynomial regression models of degrees 1, 3, 9, and 15 on a dataset containing 150 noisy house-area and price samples. To compare training and validation errors, estimate bias and variance through repeated resampling, and select the best polynomial degree using NumPy, Scikit-learn, and Matplotlib.

SYSTEM ARCHITECTURE



2. MATHEMATICAL FORMULAS

1. Polynomial Regression

$$\boxed{y = b_0 + b_1 x + b_2 x^2 + \cdots + b_d x^d}$$

where d is the polynomial degree.

2. Mean Squared Error (MSE)

$$\boxed{\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

3. Bias

Bias measures the difference between the average model prediction and the true value.

$$\boxed{\text{Bias}(x) = E[\hat{f}(x)] - f(x)}$$

Squared bias:

$$\boxed{\text{Bias}^2(x) = (E[\hat{f}(x)] - f(x))^2}$$

4. Variance

Variance measures how much predictions change across different training samples.

$$\boxed{\text{Variance}(x) = E[(\hat{f}(x) - E[\hat{f}(x)])^2]}$$

5. Bias–Variance Decomposition

For squared-error regression:

$$\boxed{\text{Expected Error} = \text{Bias}^2 + \text{Variance} + \text{Noise}}$$

6. Training and Validation Error

$$\boxed{E_{\text{train}} = \frac{1}{n_{\text{train}}}\sum_{i=1}^{n_{\text{train}}}(y_i - \hat{y}_i)^2}$$

$$\boxed{E_{\text{validation}} = \frac{1}{n_{\text{validation}}}\sum_{i=1}^{n_{\text{validation}}}(y_i - \hat{y}_i)^2}$$

CODE

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# 1. Generate 150 noisy house-area and price samples
np.random.seed(42)

n = 150
X = np.random.uniform(500, 3000, n).reshape(-1, 1)

# True nonlinear relationship
def true_function(x):
    return 30 + 0.025*x + 0.000004*x**2

y_true = true_function(X.ravel())
y = y_true + np.random.normal(0, 12, n)

# 2. Split into training and validation data
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.25, random_state=42
)

degrees = [1, 3, 9, 15]
models = {}
results = []

# 3. Fit polynomial regression models
for degree in degrees:
    model = make_pipeline(
        PolynomialFeatures(degree=degree, include_bias=False),
        StandardScaler(),
        LinearRegression()
    )

    model.fit(X_train, y_train)
    models[degree] = model

    train_pred = model.predict(X_train)
    val_pred = model.predict(X_val)

    train_mse = mean_squared_error(y_train, train_pred)
    val_mse = mean_squared_error(y_val, val_pred)

    results.append({
        "Degree": degree,
        "Training MSE": train_mse,
        "Validation MSE": val_mse
    })
```

```
results_df = pd.DataFrame(results)

print("Training and Validation Errors:")
print(results_df.round(3).to_string(index=False))

# 4. Plot fitted curves over the data
X_plot = np.linspace(500, 3000, 400).reshape(-1, 1)

plt.figure(figsize=(12, 7))
plt.scatter(X_train, y_train, label="Training data", alpha=0.65)
plt.scatter(X_val, y_val, label="Validation data", alpha=0.65)

plt.plot(
    X_plot, true_function(X_plot.ravel()),
    "k--", linewidth=2, label="True relationship"
)

for degree in degrees:
    plt.plot(
        X_plot,
        models[degree].predict(X_plot),
        label=f"Degree {degree}"
    )

plt.xlabel("House Area (sq. ft.)")
plt.ylabel("House Price (illustrative units)")
plt.title("Polynomial Regression: Underfitting vs Overfitting")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

# 5. Plot training and validation errors
plt.figure(figsize=(8, 5))
plt.plot(
    results_df["Degree"],
    results_df["Training MSE"],
    marker="o", label="Training MSE"
)
plt.plot(
    results_df["Degree"],
    results_df["Validation MSE"],
    marker="s", label="Validation MSE"
)

plt.xlabel("Polynomial Degree")
plt.ylabel("Mean Squared Error")
plt.title("Training and Validation Error vs Model Complexity")
plt.xticks(degrees)
plt.legend()
plt.grid(alpha=0.3)
plt.show()

# 6. Estimate bias squared and variance using repeated resampling
# Use a fixed test grid and the known true function.
X_grid = np.linspace(500, 3000, 100).reshape(-1, 1)
y_grid_true = true_function(X_grid.ravel())
```

```
rng = np.random.default_rng(123)
n_repeats = 100
bias_variance_results = []

for degree in degrees:
    predictions = []

    for _ in range(n_repeats):
        # Resample 150 observations with replacement
        indices = rng.choice(n, size=n, replace=True)
        X_boot = X[indices]
        y_boot = y[indices]

        model = make_pipeline(
            PolynomialFeatures(
                degree=degree, include_bias=False
            ),
            StandardScaler(),
            LinearRegression()
        )

        model.fit(X_boot, y_boot)
        predictions.append(model.predict(X_grid))

    predictions = np.array(predictions)

    mean_prediction = predictions.mean(axis=0)

    bias_squared = np.mean(
        (mean_prediction - y_grid_true)**2
    )

    variance = np.mean(
        np.var(predictions, axis=0)
    )

    bias_variance_results.append({
        "Degree": degree,
        "Bias Squared": bias_squared,
        "Variance": variance
    })

bv_df = pd.DataFrame(bias_variance_results)

print("\nBias-Variance Results:")
print(bv_df.round(3).to_string(index=False))

# 7. Plot bias squared and variance
plt.figure(figsize=(8, 5))
plt.plot(
    bv_df["Degree"], bv_df["Bias Squared"],
    marker="o", label="Squared Bias"
)
plt.plot(
    bv_df["Degree"], bv_df["Variance"],
    marker="s", label="Variance"
)
```

```
plt.xlabel("Polynomial Degree")
plt.ylabel("Estimated Error Component")
plt.title("Bias-Variance Analysis")
plt.xticks(degrees)
plt.legend()
plt.grid(alpha=0.3)
plt.show()

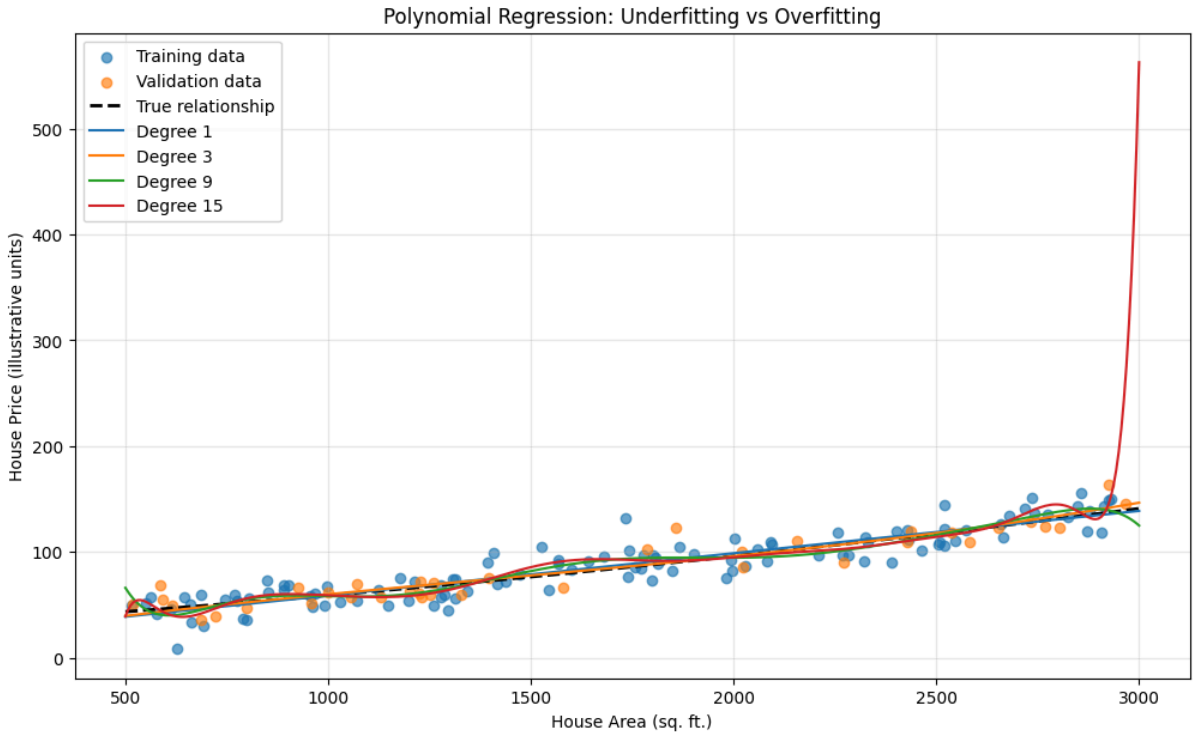
# 8. Select the best degree using validation MSE
best_row = results_df.loc[
    results_df["Validation MSE"].idxmin()
]

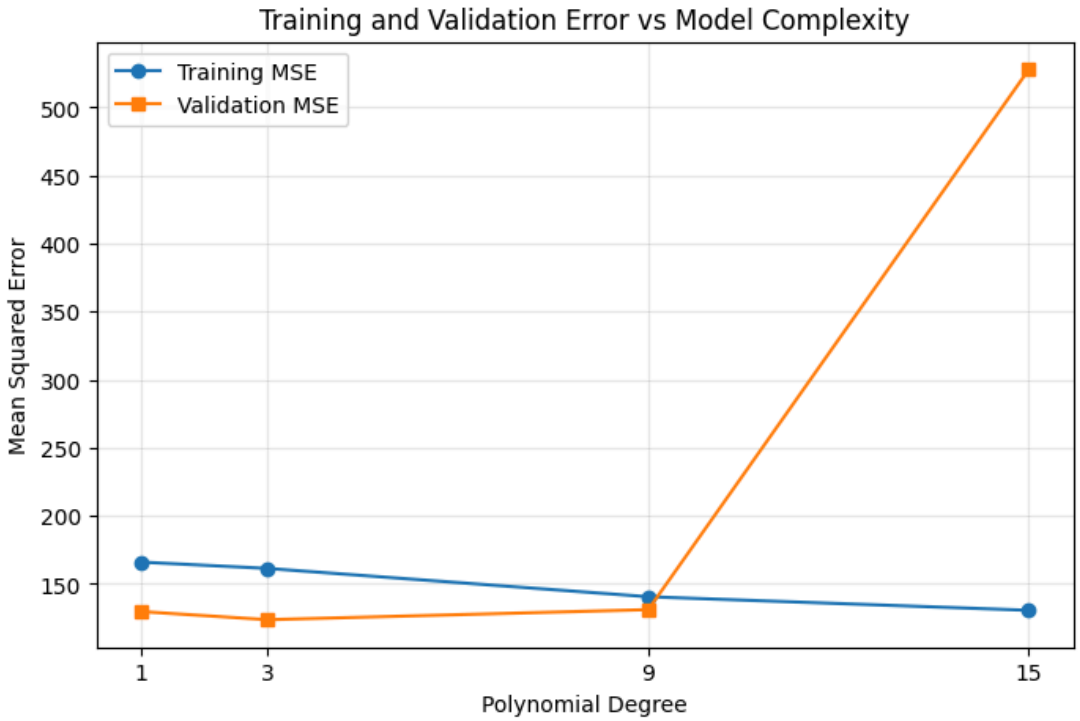
print("\nBest Polynomial Degree:", int(best_row["Degree"]))
print("Lowest Validation MSE:", round(
    best_row["Validation MSE"], 3
))
```

OUTPUT

raining and Validation Errors:

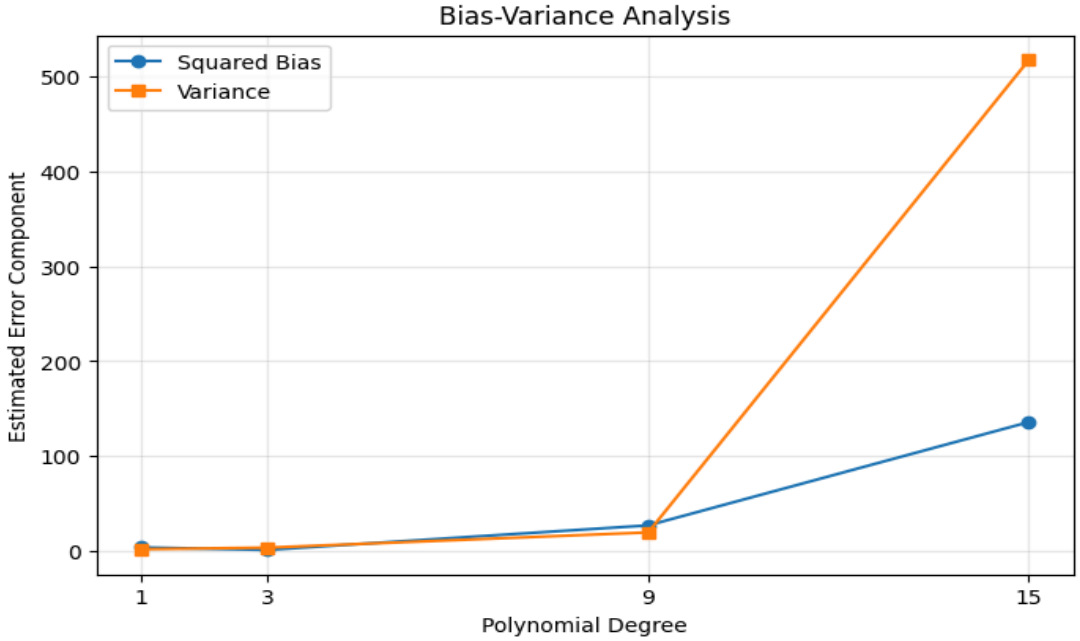
Degree	Training MSE	Validation MSE
1	166.037	129.699
3	161.370	123.704
9	140.580	131.100
15	130.616	527.916





Bias-Variance Results:

Degree	Bias Squared	Variance
1	4.119	2.182
3	1.495	3.885
9	27.305	19.919
15	136.000	517.022



Best Polynomial Degree: 3
Lowest Validation MSE: 123.704

5. RESULT

Polynomial regression models of degrees 1, 3, 9, and 15 were implemented using Scikit-learn. Their fitted curves, training and validation errors, and estimated squared bias and variance were compared using Matplotlib. The analysis demonstrates the trade-off between underfitting and overfitting. The best polynomial degree is selected using the lowest validation mean squared error, providing a data-driven approach to choosing an appropriate model complexity.